```
def get_qtarget_ddqn(qvals,r,df,done):          3
    targets = r + (1-done) * df * qvals
    return targets
```

- ***1*** Normalizes the state tensor and converts to PyTorch tensor
- ***2*** Gets a random mini-batch from the experience replay memory
- ***3*** Calculates the target Q value

These functions just prepare the state observation tensor, produce a mini-batch and calculate the target Q value as we discussed earlier.

In listing 10.9 we define the loss function we will use and also a function to update the experience replay.

Listing 10.9. Loss function and updating the replay

```
def lossfn(pred,targets,actions):           1
    loss = torch.mean(torch.pow(\
                                targets.detach() -\

pred.gather(dim=1,index=actions.unsqueeze(dim=1)).squeeze()\
                                ,2),dim=0)
    return loss

def update_replay(replay,exp,replay_size):  2
  r = exp[2]
  N = 1
  if r > 0:
      N = 50
  for i in range(N):
      replay.append(exp)
  return replay

action_map = {                              3
    0:0,
    1:1,
    2:2,
    3:3,
    4:5,
}
```

- ***1*** Loss function
- ***2*** Adds new experiences to the experience replay memory; if the reward is positive, we add 50 copies of the memory.